

# Computer Vision Project

Optimization and Machine Learning (151-0840-00L)

TA contact: Francesco Scipione ([fscipion@ethz.ch](mailto:fscipion@ethz.ch))

## 1 Project Overview

This project focuses on building and analyzing Convolutional Neural Networks (CNNs) for binary image classification of mechanical test specimens with or without fractures.

### Reference (Dataset):

- Adrien Müller et al. (2021). “Machine Learning Classifiers for Surface Crack Detection in Fracture Experiments.”

You will work with a dataset of **grayscale microscopy images** ( $128 \times 128$  pixels) from three categories:

- **ASB** — 3,741 samples (DO NOT USE THIS DATASET<sup>1</sup>)
- **NT** — 1,407 samples
- **UT** — 881 samples

Each category contains images labeled as either **class 0** or **class 1** (e.g., absence/presence of a fracture).

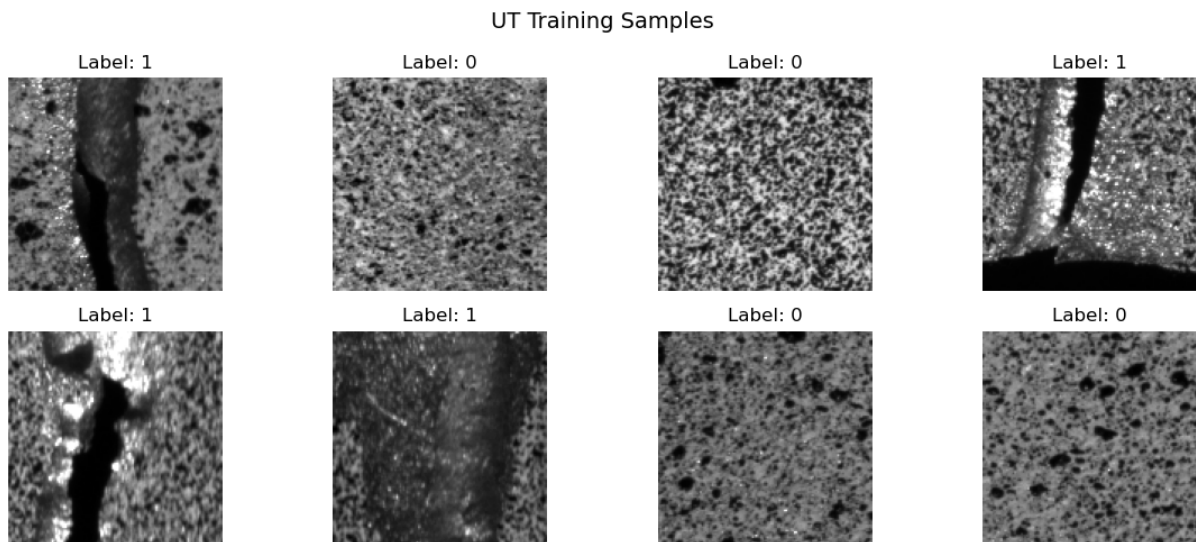


Figure 1: Samples from the UT dataset. Labels are 0 for "no fracture", and 1 for "fracture".

---

<sup>1</sup>We realized that this dataset has wrongly annotated labels and advise you not to use this dataset. You are free to test your classifier on this dataset and visually inspect the results, but the annotated labels have systematic problems we are trying to solve.

## 1.1 Learning Objectives

By completing this project, you will:

- Design and train CNN architectures for image classification
- Apply hyperparameter optimization techniques
- Analyze model robustness and failure modes
- Interpret what neural networks learn through visualization techniques
- Compare different deep learning architectures

## 2 Grading Structure

| Task          | Description                              | Points                 |
|---------------|------------------------------------------|------------------------|
| <b>Task 0</b> | Build a Simple CNN Classifier            | <b>0.05</b> (Required) |
| Task 1        | Hyperparameter Studies & Fine-tuning     | 0.05                   |
| Task 2        | Robustness Analysis (Gaussian Noise)     | 0.05                   |
| Task 3        | Feature Visualization & Interpretability | 0.05                   |
| Task 4        | Confusion Matrix Analysis                | 0.05                   |
| Task 5        | Cross-Dataset Generalization             | 0.05                   |
| Task 6        | Architecture Comparison (CNN vs ViT)     | 0.05                   |

Table 1: Task overview and point allocation

### Important Rules

1. **Task 0 is mandatory** — This is the foundation for all other tasks.
2. **Choose any 4 additional tasks** from Tasks 1–6 to achieve the maximum score.
3. **Maximum score: 0.25 points** (Task 0 + 4 optional tasks).
4. **Tasks are Pass/Fail** — No partial credit within a task.
5. **Code is required** — If code is missing or cannot reproduce results, the task is **voided**.
6. **Extra tasks as backup** — You may complete more than 5 tasks total; extra passed tasks can compensate for failed ones (still capped at 0.25).
7. **Custom tasks** — Additional tasks beyond Tasks 1–6 must be **discussed and approved in advance**.
8. **Report Length** — A maximum of 2 pages per task is allowed. If this limit is exceeded, the task is **voided**. It is possible to include an appendix with additional material for completeness but you will be evaluated on the main report.
9. **Collaboration** — You are allowed (and encouraged) to collaborate. Nevertheless, the submission is individual. You are allowed to share codes and results between teammates and append to your personal submission.

### 3 Project Organization

Your submission must follow this folder structure:

```

OptML_CNN_project/
|-- PROJECT_INSTRUCTIONS.md      # This document
|-- requirements.txt             # Python dependencies
|
|-- data/
|   |-- mmc1/                   # Original .mat files (provided)
|   +-- processed/              # NPZ files (provided)
|       |-- ASB/
|           |-- train.npz
|           |-- val.npz
|           +-- test.npz
|       |-- NT/
|       +-- UT/
|
|-- notebooks/
|   +-- nb_beginner_guide.ipynb # Data exploration (provided)
|
|-- src/
|   +-- (shared utilities)
|
|-- tasks/                      # <-- YOUR WORK GOES HERE
|   |-- task_0_baseline_cnn/
|   |   |-- train.py            # Training script
|   |   |-- model.py            # Model definition
|   |   |-- results/            # Saved models, metrics
|   |   +-- README.md           # Task description & results
|   |
|   |-- task_1_hyperparameter/
|   |-- task_2_robustness/
|   |-- task_3_interpretability/
|   |-- task_4_confusion_matrix/
|   |-- task_5_cross_dataset/
|   +-- task_6_cnn_vs_vit/
|
+-- report/
    |-- report.pdf              # Final report
    +-- figures/                # Report figures
  
```

#### 3.1 Task Folder Requirements

Each `task_X_*/` folder **must contain**:

1. **Python scripts** — Runnable code that reproduces your results
2. **README.md** — Brief description of approach and key findings
3. **Results** — Figures, metrics, saved models

### 4 Task Descriptions

#### 4.1 Task 0: Build a Simple CNN Classifier (Required)

**Objective:** Implement and train a basic CNN to classify images from one of the datasets (NT, or UT).

**Requirements:**

- Design a CNN architecture with at least 2–3 convolutional layers

- Implement a training loop with train/validation split
- Report final test accuracy
- Plot training and validation loss curves

**Suggested Starting Architecture:**

```
Conv2d(1, 32, 3) -> ReLU -> MaxPool2d(2)
Conv2d(32, 64, 3) -> ReLU -> MaxPool2d(2)
Conv2d(64, 128, 3) -> ReLU -> AdaptiveAvgPool2d(1)
Linear(128, 2)
```

**Objectives**

- `model.py` — CNN architecture definition
- `train.py` — Training script
- Training curves plot
- Test accuracy on held-out test set

**References:**

- LeCun, Y., et al. (1998). “Gradient-based learning applied to document recognition.” *Proceedings of the IEEE*.
- Krizhevsky, A., et al. (2012). “ImageNet Classification with Deep Convolutional Neural Networks.” *NeurIPS*.

## 4.2 Task 1: Hyperparameter Studies & Fine-tuning

**Objective:** Systematically explore how hyperparameters affect model performance.

**Options (choose one):**

### 4.2.1 Option A: Grid/Random Search

- Search over learning rate, batch size, optimizer, architecture depth
- Create a table/heatmap of results
- Identify the best configuration

### 4.2.2 Option B: Optuna Implementation

- Implement automated hyperparameter optimization using Optuna
- Define a search space and objective function
- Visualize optimization history and parameter importance

**Objectives**

- Search script (`grid_search.py` or `optuna_search.py`)
- Results table/visualization
- Analysis of which hyperparameters matter most

**References:**

- Bergstra, J., & Bengio, Y. (2012). “Random Search for Hyper-Parameter Optimization.” *JMLR*.
- Akiba, T., et al. (2019). “Optuna: A Next-generation Hyperparameter Optimization Framework.” *KDD*.

### 4.3 Task 2: Robustness Analysis (Gaussian Noise)

**Objective:** Analyze how model accuracy degrades when images are corrupted with Gaussian noise.

**Requirements:**

- Add Gaussian noise with varying  $\sigma$  (standard deviation) values: e.g.,  $\sigma \in \{0, 0.05, 0.1, 0.15, 0.2, 0.3, 0.5\}$
- Evaluate model accuracy at each noise level
- Plot accuracy vs. noise level curve
- Discuss: At what noise level does the model fail? Why?

**Noise Addition:**

```
1 def add_gaussian_noise(images, sigma):  
2     noise = torch.randn_like(images) * sigma  
3     return torch.clamp(images + noise, 0, 1)
```

#### Objectives

- Noise analysis script
- Accuracy vs.  $\sigma$  plot
- Example images at different noise levels
- Discussion of robustness characteristics

**References:**

- Hendrycks, D., & Dietterich, T. (2019). “Benchmarking Neural Network Robustness to Common Corruptions and Perturbations.” *ICLR*.
- Dodge, S., & Karam, L. (2017). “A Study and Comparison of Human and Deep Learning Recognition Performance Under Visual Distortions.” *ICCCN*.

### 4.4 Task 3: Feature Visualization & Interpretability

**Objective:** Understand what the CNN learns by visualizing filters, activations, and attention maps.

**Choose at least 2 of the following:**

#### 4.4.1 A. Filter Visualization

- Visualize learned convolutional filters at different layers
- What patterns do early vs. late layers detect?

#### 4.4.2 B. Activation Maps

- Show feature maps for sample images at each layer
- How does the representation change through the network?

#### 4.4.3 C. Grad-CAM

- Implement Gradient-weighted Class Activation Mapping
- Highlight which image regions influence the prediction

#### 4.4.4 D. t-SNE/UMAP Embeddings

- Extract features from the penultimate layer
- Visualize 2D embeddings colored by class

##### Key Questions to Address:

- What shapes/patterns is the CNN learning to recognize?
- How does a CNN understand what a fracture/defect looks like?
- Are the learned features interpretable?

##### Objectives

- Visualization scripts
- Figure gallery with interpretations
- Discussion of what the CNN has learned

##### References:

- Zeiler, M. D., & Fergus, R. (2014). “Visualizing and Understanding Convolutional Networks.” *ECCV*.
- Selvaraju, R. R., et al. (2017). “Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization.” *ICCV*.
- Olah, C., et al. (2017). “Feature Visualization.” *Distill*. <https://distill.pub/2017/feature-visualization/>

### 4.5 Task 4: Confusion Matrix Analysis

**Objective:** Perform detailed error analysis to understand model failures.

##### Requirements:

- Compute and visualize the confusion matrix on the test set
- Calculate precision, recall, F1-score per class
- Identify and visualize misclassified examples
- Analyze: Are there patterns in the errors? (e.g., ambiguous images, edge cases)

##### Questions to Address:

- Which class is harder to classify? Why?
- What do misclassified images have in common?
- How could you improve performance on the failure cases?

**Objectives**

- Confusion matrix visualization
- Classification report (precision, recall, F1)
- Gallery of misclassified examples with analysis
- Suggestions for improvement

**References:**

- Provost, F., & Fawcett, T. (2013). *Data Science for Business*. O'Reilly Media. (Chapter on Evaluation)

#### 4.6 Task 5: Cross-Dataset Generalization

**Objective:** Evaluate how well a model trained on one dataset generalizes to others.

**Experiments:**

1. Train on NT → Test on UT
2. Train on UT → Test on NT

**Analysis:**

- Create a generalization matrix (3×3 accuracy table)
- Which transfers work well? Which fail?
- Discuss domain shift and feature similarity

**Objectives**

- Cross-evaluation script
- Generalization matrix table
- Analysis of transfer performance

**References:**

- Torralba, A., & Efros, A. A. (2011). “Unbiased Look at Dataset Bias.” *CVPR*.
- Pan, S. J., & Yang, Q. (2010). “A Survey on Transfer Learning.” *IEEE TKDE*.

#### 4.7 Task 6: Architecture Comparison (CNN vs ViT)

**Objective:** Compare traditional CNNs with Vision Transformers on this dataset.

**Requirements:**

- Implement or use a pre-trained Vision Transformer (ViT)
- Train both CNN and ViT on the same dataset

- Compare: accuracy, training time, parameter count
- Analyze: Which works better for this small dataset? Why?

**Considerations:**

- ViTs typically need more data; consider using pre-trained models
- Discuss inductive biases of CNNs vs. ViTs

**Objectives**

- ViT implementation or adaptation script
- Comparison table (accuracy, params, FLOPs, training time)
- Analysis of when to use CNN vs. ViT

**References:**

- Dosovitskiy, A., et al. (2021). “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale.” *ICLR*.
- Liu, Z., et al. (2021). “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows.” *ICCV*.

## 5 Report Requirements

Your final report (`report/report.pdf`) must include:

### 5.1 For Each Completed Task:

1. **Task Number & Title** — Clearly state which task you are presenting
2. **Approach** — Brief description of your methodology
3. **Results** — Figures, tables, metrics
4. **Discussion** — Interpretation of results, insights gained
5. **Code Location** — Reference to the corresponding `tasks/task_X_*/` folder

## 5.2 Report Structure Example:

```
1. Introduction
  - Project overview
  - Tasks completed: 0, 1, 2, 4, 5

2. Task 0: Baseline CNN
  - Architecture description
  - Training results
  - Test accuracy: XX%

3. Task 1: Hyperparameter Optimization
  - Search methodology
  - Best configuration found
  - [...]

4. [Continue for each task]

5. Conclusion
  - Summary of findings
  - Lessons learned

References
```

## 6 Getting Started

### 6.1 Environment Setup

```
1 # Create conda environment
2 conda create -n cnn_project python=3.10
3 conda activate cnn_project
4
5 # Install dependencies
6 pip install torch torchvision numpy scipy scikit-learn matplotlib optuna
```

### 6.2 Data Loading

See notebooks/nb\_beginner\_guide.ipynb for a complete tutorial on:

- Loading NPZ files
- Creating PyTorch DataLoaders
- Visualizing samples

#### Quick Example:

```
1 import numpy as np
2 import torch
3 from torch.utils.data import TensorDataset, DataLoader
4
5 # Load data
6 data = np.load("data/processed/UT/train.npz")
7 images = torch.from_numpy(data["images"]) # (N, 1, 128, 128)
8 labels = torch.from_numpy(data["labels"]) # (N,)
9
10 # Create DataLoader
11 dataset = TensorDataset(images, labels)
12 loader = DataLoader(dataset, batch_size=32, shuffle=True)
```

## 7 FAQ

**Q: Can I use pre-trained models?**

A: Yes, especially for Task 6 (ViT). Just document what you use.

**Q: What if my code doesn't run on the grader's machine?**

A: Include a `requirements.txt` and clear instructions. Test your code in a fresh environment.

**Q: Can I collaborate with other students?**

A: You can collaborate with other students but you will submit your project individually.

**Q: What if I want to do a task not listed?**

A: Contact Francesco Scipione and propose it in advance for approval. Include clear objectives and evaluation criteria.

## 8 Additional References

### 8.1 Deep Learning Fundamentals

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- CS231n: Convolutional Neural Networks for Visual Recognition. Stanford University.

### 8.2 PyTorch Resources

- PyTorch Tutorials: <https://pytorch.org/tutorials/>
- PyTorch Documentation: <https://pytorch.org/docs/stable/index.html>

### 8.3 Interpretability

- Distill.pub — <https://distill.pub/> (Excellent visualizations of neural network concepts)
- Captum Library (PyTorch interpretability): <https://captum.ai/>

**Good luck with your project!**